

Data Structures

Lecture 7: Stack and Queue

Soobin Um

Mar. 30, 2026

공지사항: 코딩테스트

PCCE 확정 일정

사용언어별로 날짜가 다름. 장소는 모두 온라인.

1. C++

- 5/19(화) 오후8시 or 5/21(목) 오후8시 @ 온라인

2. Python

- 5/20(수) 오후8시 or 5/21(목) 오후8시 @ 온라인

3. Java

- 5/21(목) 오후8시 @ 온라인

PCCP 확정 일정

모든 언어 같은 날짜에 시험 봄. 장소는 온라인.

C++, Python, Java 중 택 1

- 5/23(토) 오후2시 @ 온라인

성적 반영: **단서조항 추가**(학과지침 $\pi\pi$)

(35%) 중간고사

(35%) 기말고사

(10%) 과제

(10%) 출석

(10%) 수업참여도(PCCE/P 응시 결과)

코딩테스트 0점인 경우, 미응시로 간주 → 기본점수 없음

기타 안내

시험 접수(학생 등록)

- 시험 접수는 학생들이 직접 개별적으로 진행.
- 시험에 대한 안내 및 접수 관련 매뉴얼은 추후 학부 홈페이지를 통해 공지할 예정.
- **타과생 응시 가능.**

시험 진행

- **온라인**이며, **프로그래머스 시스템** 사용 예정.
- 응시 일주일 전부터 하루 전까지 온라인 시험 환경 사전 체험 가능(추후 배포되는 매뉴얼 참조)

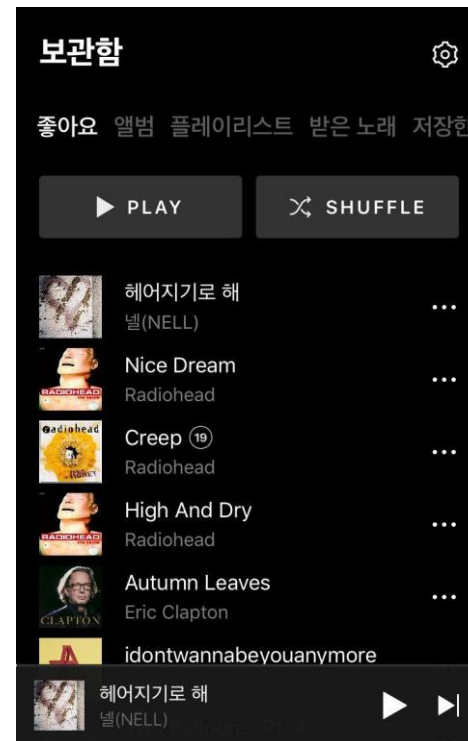
문의처

- 02-910-5988 (SW중심대학사업단 김민서)

복습

Lists (리스트)

- 순서가 있는 데이터의 나열(ordered collection)
- 리스트를 구성하는 각 데이터(원소)는 리스트 상에서 각자 자신의 위치(인덱스)를 가짐
 - 인덱스 기반 접근 가능 (예: list[0], list[1])



Key Operations

- **탐색 (Search)**
 - 특정 인덱스의 원소를 찾거나, 특정 값을 가진 원소를 찾는 것
- **갱신 (Update)**
 - 특정 인덱스의 원소를 새로운 값으로 교체
- **삽입 (Insert)**
 - 새로운 원소를 원하는 위치에 추가
 - 배열에서의 경우, 뒤의 원소들을 밀어내야 함
- **삭제 (Delete)**
 - 특정 위치의 원소를 제거
 - 배열에서의 경우, 뒤의 원소들을 앞으로 당겨야 함
- **순회 (Traverse)**
 - 리스트의 모든 원소를 차례대로 읽음
 - 예: 출력, 합계 계산

List ADT

- 구현 방법과 상관없이 인터페이스 차원에서 “**위치를 가진 원소들의 유한 순서열**”을 정의

```
public interface List<E> {  
    public void clear();  
    public void insert(int pos, E item);  
    public void append(E item);  
    public void update(int pos, E item);  
    public E getValue(int pos);  
    public E remove(int pos);  
    public void printAll();  
}
```

※ 제네릭 타입 E: Element의 약자로, 리스트에 어떤 자료형을 담을지를 **나중에 결정**

Question: ADT → Data Structure

“리스트를 실제로 어떻게 구현할 수 있을까?”
(= 리스트 상에서의 원소들을 어떻게 줄 세울 수 있을까?)

배열(array) 기반 구현

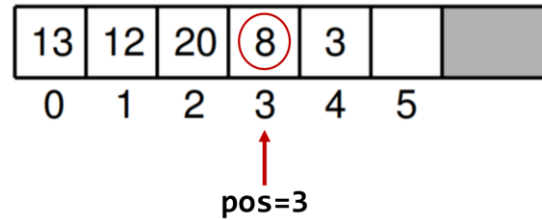
: 연속된 빈칸에 데이터를 차례대로 저장

연결 리스트(linked list) 기반 구현

: 각 칸마다 옆 칸을 가리키는 화살표를 붙여서 연결

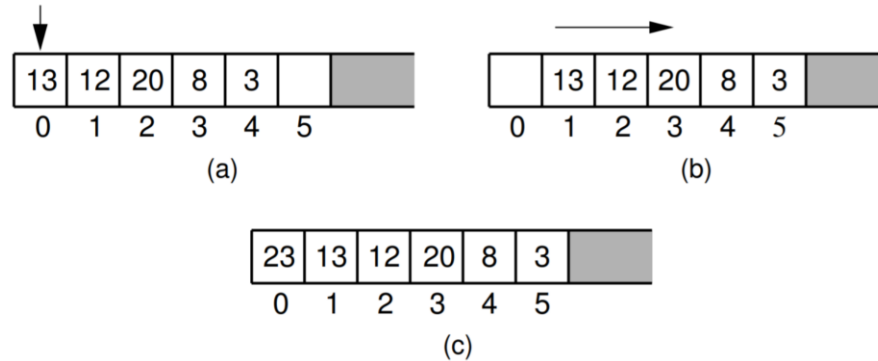
Operations: 배열 기반 리스트

- 탐색 ($\in O(1)$)

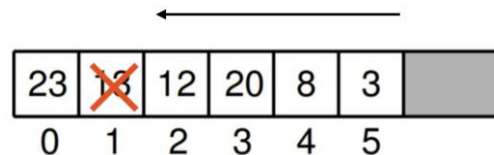


- 삽입 ($\in O(n)$)

Insert 23:

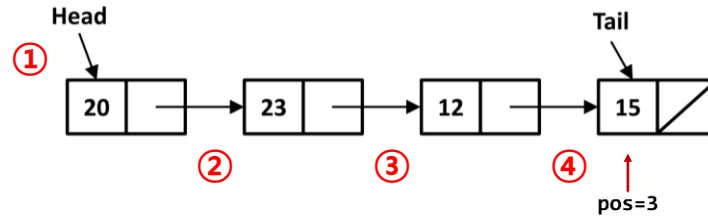


- 삭제 ($\in O(n)$)

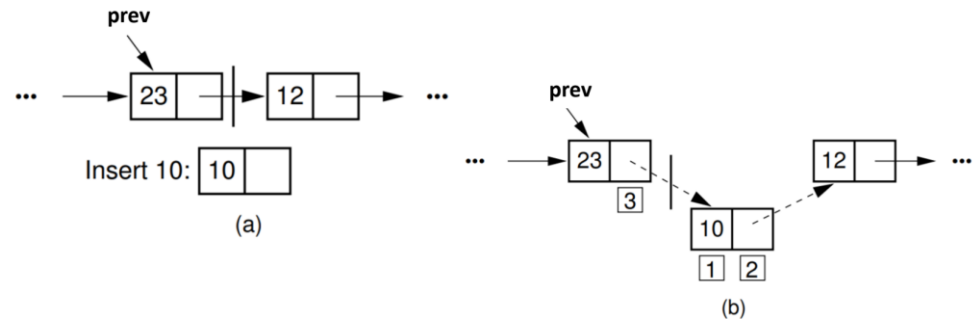
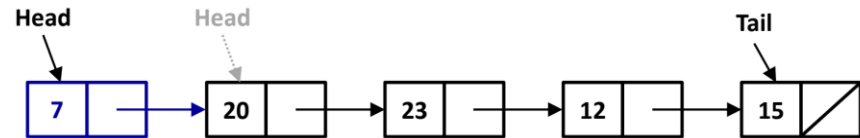


Operations: 연결 리스트

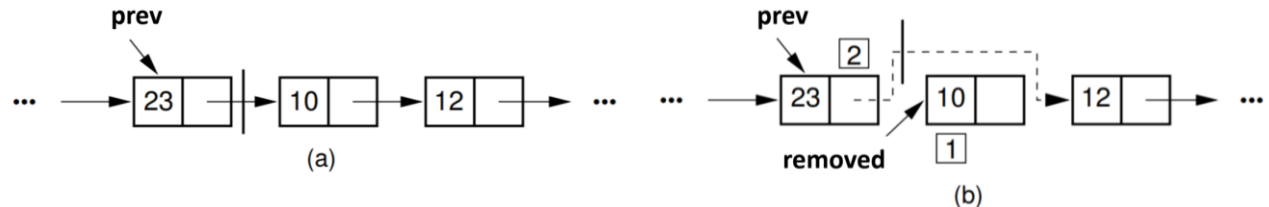
- 탐색 ($\in O(n)$)



- 삽입 ($\in O(1)$ or $O(n)$)



- 삭제 ($\in O(1)$ or $O(n)$)



배열 기반 리스트 vs. 연결 리스트

연산	배열 기반 리스트	연결 리스트	참고사항
탐색 (인덱스 접근)	$O(1)$	$O(n)$	배열은 인덱스로 즉시 접근, 연결 리스트는 순차 탐색 필요
탐색 (특정 값)	$O(n)$	$O(n)$	둘 다 순차 탐색 필요
삽입 (마지막 위치)	$O(1)$	$O(1)$	배열은 공간 여유 시 $O(1)$, 연결 리스트는 tail 사용 시 $O(1)$
삽입 (지정 위치)	$O(n)$	$O(n)$ (접근) + $O(1)$ (링크 변경)	배열은 원소 이동 필요, 연결 리스트는 이전 노드까지 탐색
삭제 (마지막 위치)	$O(1)$	$O(n)$	배열은 사이즈만 줄이면 됨, 연결 리스트는 tail 이전 노드 탐색 필요
삭제 (지정 위치)	$O(n)$	$O(n)$ (접근) + $O(1)$ (링크 변경)	연결 리스트 삭제 연산 자체는 $O(1)$, but 탐색은 $O(n)$
순회	$O(n)$	$O(n)$	둘 다 순차 탐색 필요

스택 Stack

스택 Stack

- 자료를 **쌓아 올린** 형태로 데이터를 저장하는 자료 구조

스택 Stack

- 자료를 쌓아 올린 형태로 데이터를 저장하는 자료 구조
- 가장 나중에 삽입된 데이터가 가장 먼저 삭제되는 후입선출 (LIFO, Last-In First-Out) 구조
 - 즉, 삽입 및 삭제가 리스트의 한쪽 끝에서만 가능한, 리스트의 제한된 형태와도 같음

스택 Stack

- 자료를 **쌓아 올린** 형태로 데이터를 저장하는 자료 구조
- 가장 나중에 삽입된 데이터가 가장 먼저 삭제되는 **후입선출 (LIFO, Last-In First-Out)** 구조
 - 즉, **삽입 및 삭제가 리스트의 한쪽 끝에서만 가능**한, 리스트의 제한된 형태와도 같음



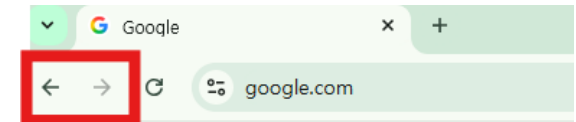
스택 Stack

- 자료를 **쌓아 올린** 형태로 데이터를 저장하는 자료 구조
- 가장 나중에 삽입된 데이터가 가장 먼저 삭제되는 **후입선출 (LIFO, Last-In First-Out)** 구조
 - 즉, **삽입 및 삭제가 리스트의 한쪽 끝에서만 가능**한, 리스트의 제한된 형태와도 같음



스택 Stack

- 자료를 **쌓아 올린** 형태로 데이터를 저장하는 자료 구조
- 가장 나중에 삽입된 데이터가 가장 먼저 삭제되는 **후입선출 (LIFO, Last-In First-Out)** 구조
 - 즉, **삽입 및 삭제가 리스트의 한쪽 끝에서만 가능**한, 리스트의 제한된 형태와도 같음



구조 및 주요 연산

- 구조
 - 리스트의 한쪽 끝(top)에서만 삽입 또는 삭제 연산이 이뤄짐

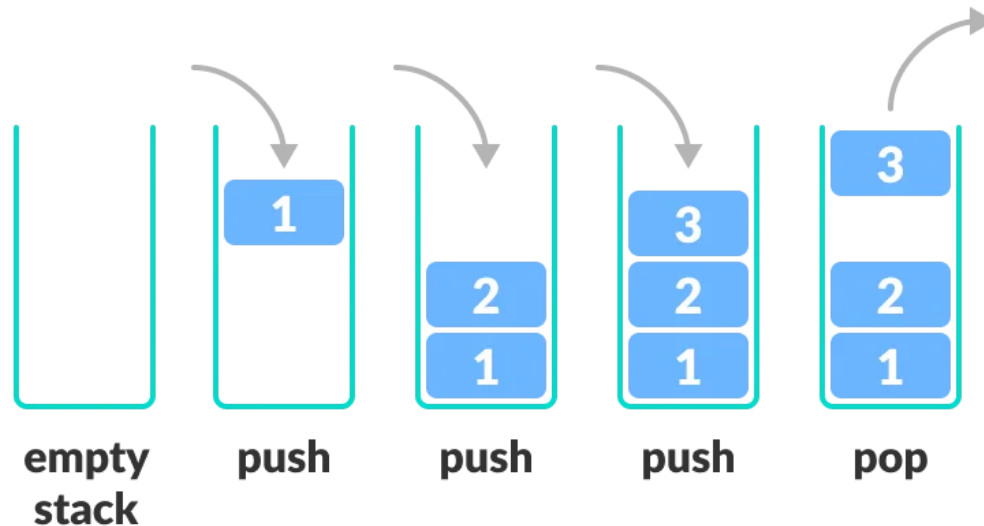
구조 및 주요 연산

- 구조

- 리스트의 한쪽 끝(top)에서만 삽입 또는 삭제 연산이 이뤄짐

- 주요 연산

- **push(item)**: 스택의 맨 위에 원소(item)를 삽입
- **pop()**: 스택의 맨 위 원소를 꺼내어 제거
- **topValue()**: 스택의 맨 위 원소를 제거하지 않고 확인 (리턴)



Stack ADT

- 구현 방법과 무관하게 스택이 반드시 제공해야 하는 동작들의 집합

```
public interface Stack<E> {  
    /** Reinitialize the stack. */  
    public void clear();  
  
    /** Push an element onto the top of the stack.  
    @param it The element being pushed onto the stack. */  
    public void push(E it);  
  
    /** Remove and return the element at the top of the stack.  
    @return The element at the top of the stack. */  
    public E pop();  
  
    /** @return A copy of the top element. */  
    public E topValue();  
  
    /** @return The number of elements in the stack. */  
    public int length();  
}
```

Question: ADT $\rightarrow$ Data Structure

인터페이스 Stack<E>에 대한 구현 방법?

Question: ADT $\rightarrow$ Data Structure

인터페이스 $\text{Stack}\langle E \rangle$ 에 대한 구현 방법?

배열 기반 구현 (ArrayStack)

: 연속된 빈칸에 데이터를 차례대로 저장

Question: ADT $\rightarrow$ Data Structure

인터페이스 $\text{Stack}\langle E \rangle$ 에 대한 구현 방법?

배열 기반 구현 (ArrayStack)

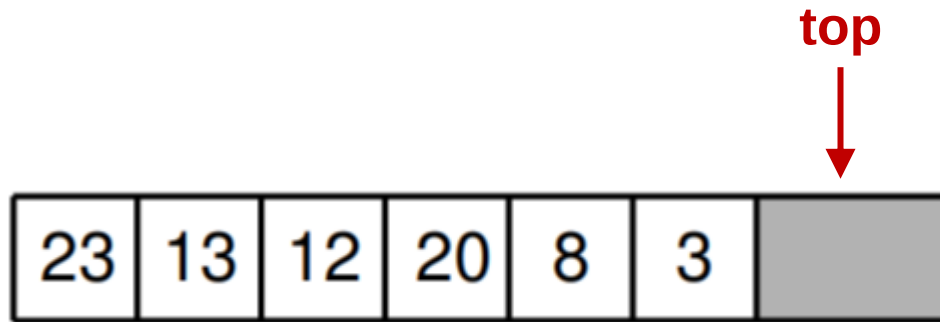
: 연속된 빈칸에 데이터를 차례대로 저장

연결 리스트 기반 구현 (LinkedStack)

: 각 칸마다 옆 칸을 가리키는 화살표를 붙여서 연결

ArrayStack: 구조

- 내부에 **고정된 크기의 배열**을 둔 상태에서,
- 인덱스 변수(top)를 통해 현재 스택의 **맨 위 위치를 추적**



ArrayStack: 코드

```
public class ArrayStack<E> implements Stack<E> {
    private E[] listArray;    // 스택 원소 저장
    private int top;          // 스택 맨 위 인덱스
    private int capacity;

    public ArrayStack(int size) {
        listArray = (E[]) new Object[size]; // Object 배열 생성 후 E 배열로 변환
        capacity = size;
        top = 0;
    }

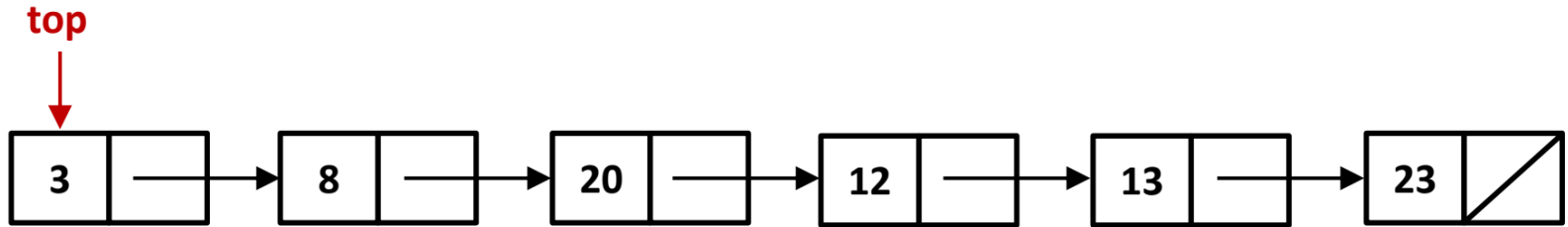
    public void push(E it) {
        if (top == capacity) throw new RuntimeException("Stack is full");
        listArray[top++] = it;
    }

    public E pop() {
        ...    // 각자 완성해보기
    }

    public E topValue() {
        ...    // 각자 완성해보기
    }
}
```

LinkedStack: 구조

- 스택의 각 원소를 노드로 구성
 - 각 노드는 데이터(item)와 다음 노드 참조(next)를 가짐
- 스택의 top은 리스트의 맨 앞 노드를 가리킴



LinkedList: 코드

```
public class LinkedList<E> implements Stack<E> {
    private static class Node<E> {
        ...      // e.g., 연결 리스트 때 사용한 노드 클래스
    }

    private Node<E> top;  // 스택 맨 위 노드
    private int size;

    public LinkedList() {
        top = null;
        size = 0;
    }

    public void push(E it) {
        top = new Node<>(it, top); // next node를 현재의 top으로
        size++;
    }

    public E pop() {
        ...      // 각자 완성해보기
    }

    public E topValue() {
        ...      // 각자 완성해보기
    }
}
```

Time Complexity (시간 복잡도)

- 일반적으로, 두 구현 방식 모두 모든 연산이 상수 시간 $O(1)$ 내 수행 가능

연산	ArrayStack	LinkedStack	참고사항
push	$O(1)$	$O(1)$	ArrayStack의 경우, 배열(listArray)이 가득차면 $O(n)$ 크기의 재할당 필요
pop	$O(1)$	$O(1)$	ArrayStack은 배열의 top 인덱스 감소, LinkedStack은 top 노드 제거
topValue	$O(1)$	$O(1)$	
length	$O(1)$	$O(1)$	ArrayStack은 top 인덱스 값, LinkedStack은 size 변수 유지
clear	$O(1)$	$O(1)$	

Space Complexity (공간 복잡도)

- **ArrayStack:** 배열 크기만큼의 공간만 필요
 - 단, 배열 크기 초과 시, **재할당 비용** 발생
- **LinkedStack:** 각 노드마다 포인터 공간이 추가적으로 필요

ArrayStack	LinkedStack
$O(n)$	$O(n)$ + 추가적인 메모리(포인터)

큐 Queue

큐 Queue

- 줄 또는 대기열 형태로 데이터를 저장하는 자료 구조

큐 Queue

- 줄 또는 대기열 형태로 데이터를 저장하는 자료 구조
- 먼저 삽입된 데이터가 가장 먼저 삭제되는 선입선출 (FIFO, First-In First-Out) 구조
 - 즉, 한 쪽 끝에서만 삽입
 - 다른 쪽 끝에서만 삭제가 가능한, 리스트의 제한된 형태와도 같음

큐 Queue

- 줄 또는 대기열 형태로 데이터를 저장하는 자료 구조
- 먼저 삽입된 데이터가 가장 먼저 삭제되는 선입선출 (FIFO, First-In First-Out) 구조
 - 즉, 한 쪽 끝에서만 삽입
 - 다른 쪽 끝에서만 삭제가 가능한, 리스트의 제한된 형태와도 같음



큐 Queue

- 줄 또는 대기열 형태로 데이터를 저장하는 자료 구조
- 먼저 삽입된 데이터가 가장 먼저 삭제되는 선입선출 (FIFO, First-In First-Out) 구조
 - 즉, 한 쪽 끝에서만 삽입
 - 다른 쪽 끝에서만 삭제가 가능한, 리스트의 제한된 형태와도 같음



구조 및 주요 연산

- 구조

- 리스트 상에 **두 개의 끝**이 존재
 - **Rear (뒤):** 데이터를 삽입하는 쪽
 - **Front (앞):** 데이터를 삭제하는 쪽

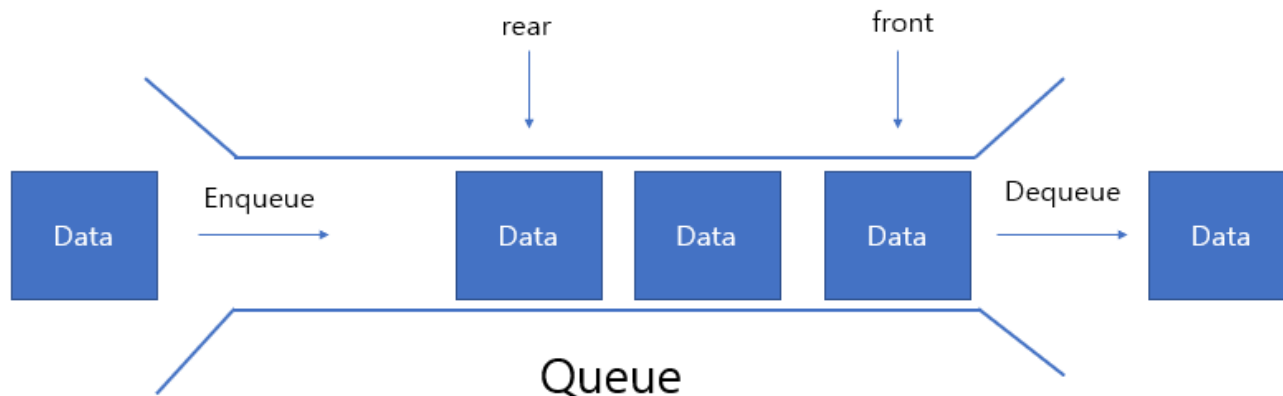
구조 및 주요 연산

- 구조

- 리스트 상에 **두 개의 끝**이 존재
 - **Rear (뒤):** 데이터를 삽입하는 쪽
 - **Front (앞):** 데이터를 삭제하는 쪽

- 주요 연산

- **enqueue(item):** 큐의 뒤쪽에 원소(item)를 삽입
- **dequeue():** 큐의 앞쪽에서 원소를 꺼내어 제거
- **frontValue():** 큐의 맨 앞 원소를 제거하지 않고 확인 (리턴)



Queue ADT

- 구현 방법과 무관하게 큐가 반드시 제공해야 하는 동작들의 집합

```
public interface Queue<E> {  
    /** Reinitialize the queue. The user is responsible for reclaiming  
    the storage used by the queue elements. */  
    public void clear();  
  
    /** Place an element at the rear of the queue.  
    @param it The element being enqueued. */  
    public void enqueue(E it);  
  
    /** Remove and return element at the front of the queue.  
    @return The element at the front of the queue. */  
    public E dequeue();  
  
    /** @return The front element. */  
    public E frontValue();  
  
    /** @return The number of elements in the queue. */  
    public int length();  
}
```


Question: ADT $\rightarrow$ Data Structure

인터페이스 Queue<E>에 대한 구현 방법?

Question: ADT $\rightarrow$ Data Structure

인터페이스 Queue<E>에 대한 구현 방법?

배열 기반 구현 (ArrayQueue)

: 연속된 빈칸에 데이터를 차례대로 저장

연결 리스트 기반 구현 (LinkedQueue)

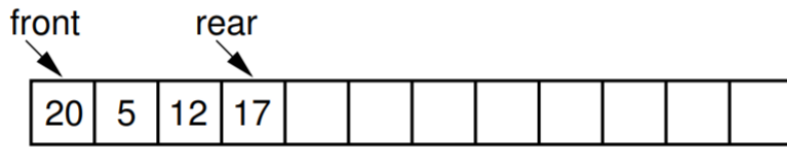
: 각 칸마다 옆 칸을 가리키는 화살표를 붙여서 연결

ArrayQueue: 구조

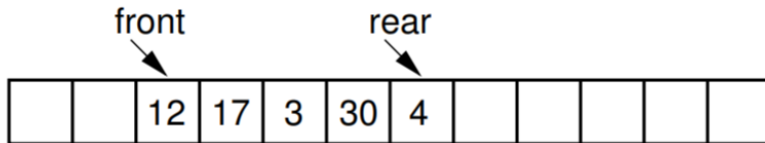
- 내부에 고정된 크기의 배열 존재
- 배열을 원형 큐 (circular queue) 구조로 구현
 - 공간 낭비 방지

ArrayQueue: 구조

- 내부에 **고정된 크기**의 배열 존재
- 배열을 **원형 큐 (circular queue)** 구조로 구현
 - 공간 낭비 방지



(a)

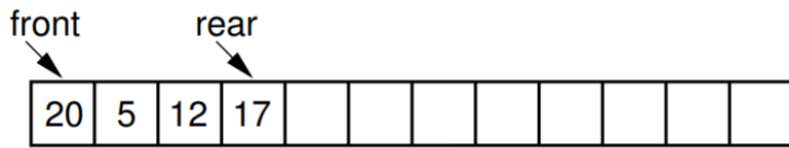


(b)

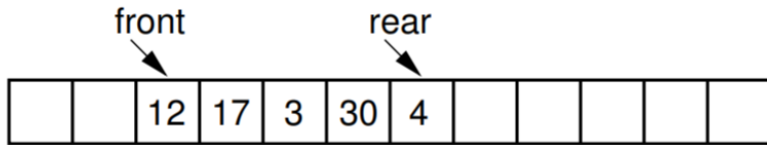
원형 큐 구조가 아닐 경우:
사용할수록 **배열의 한쪽 끝으로 점점 이동하게 됨**

ArrayQueue: 구조

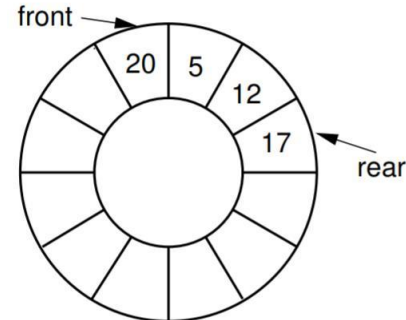
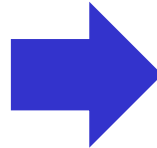
- 내부에 **고정된 크기**의 배열 존재
- 배열을 **원형 큐 (circular queue)** 구조로 구현
 - 공간 낭비 방지



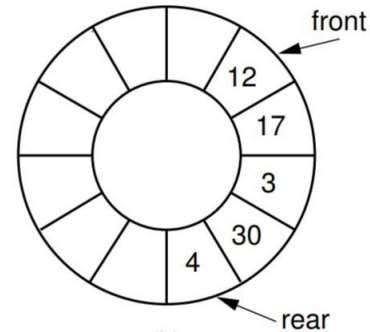
(a)



(b)



(a)



(b)

원형 큐 구조가 아닐 경우:
사용할수록 **배열의 한쪽 끝으로 점점 이동하게 됨**

원형 큐 구조

ArrayQueue: 코드

```
public class ArrayQueue<E> implements Queue<E> {
    private E[] listArray;    // 큐 원소 저장
    private int front;        // 삭제 위치
    private int rear;         // 삽입 위치
    private int size;         // 원소 개수
    private int capacity;     // 배열 용량

    public ArrayQueue(int capacity) {
        listArray = (E[]) new Object[capacity];
        this.capacity = capacity;
        front = 0; rear = 0; size = 0;
    }

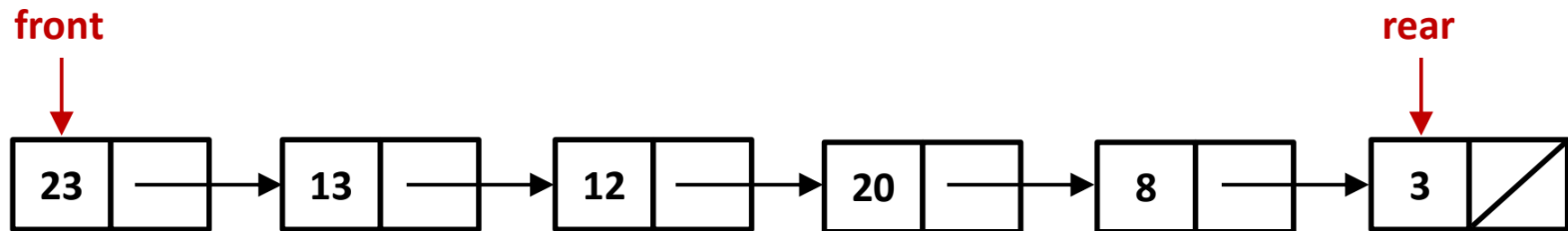
    public void enqueue(E it) {    // 큐에 새 item 삽입
        if (size == capacity) throw new RuntimeException("Queue is full");
        listArray[rear] = it;
        rear = (rear+1) % capacity;    // circular queue 방식으로 rear 인덱스 업데이트
        size++;
    }

    public E dequeue() {
        ...    // 각자 완성해보기
    }

    public E frontValue() {
        ...    // 각자 완성해보기
    }
}
```

LinkedList: 구조

- 큐의 각 원소를 노드로 구성
 - 각 노드는 데이터(item)와 다음 노드 참조(next)를 가짐
- 큐의 rear 포인터에서 삽입, front 포인터에서 삭제 연산 수행



LinkedList: 코드

```
public class LinkedList<E> implements Queue<E> {
    private static class Node<E> {
        ...      // e.g., 연결 리스트 때 사용한 노드 클래스
    }

    private Node<E> front;    // 큐의 맨 앞 노드 (삭제 위치)
    private Node<E> rear;    // 큐의 맨 뒤 노드 (삽입 위치)
    private int size;

    public LinkedList() {
        front = rear = null;
        size = 0;
    }

    public void enqueue(E it) {    // 큐에 새 노드 삽입
        Node<E> node = new Node<>(it, null);
        if (rear == null) // 큐가 비어있을 때
            front = rear = node;
        else {
            rear.next = node;
            rear = node;    // rear를 새 노드로 업데이트
        }
        size++;
    }

    public E dequeue() {...}    // 각자 완성해보기
    public E frontValue() {...}    // 각자 완성해보기
}
```


Time Complexity (시간 복잡도)

- 일반적으로, 두 구현 방식 모두 모든 연산이 상수 시간 $O(1)$ 내 수행 가능

연산	ArrayQueue	LinkedQueue	참고사항
enqueue	$O(1)$	$O(1)$	ArrayQueue의 경우, 배열(listArray)이 가득차면 $O(n)$ 크기의 재할당 필요
dequeue	$O(1)$	$O(1)$	삭제는 front 위치에서 수행
frontValue	$O(1)$	$O(1)$	
Length	$O(1)$	$O(1)$	
Clear	$O(1)$	$O(1)$	

Space Complexity (공간 복잡도)

- **ArrayQueue:** 배열 크기만큼의 공간만 필요
 - 단, 배열 크기 초과 시, **재할당 비용** 발생
- **LinkedList:** 각 노드마다 포인터 공간이 추가적으로 필요

ArrayQueue	LinkedList
$O(n)$	$O(n)$ + 추가적인 메모리(포인터)

Look ahead

Doubly Linked List, Dictionary